Introducing Pandas Objects

Python Data Science Handbook — Chapter 3, Section 3.1

Series, DataFrame, and Index — the three data structures everything else is built on.

1 Where Pandas comes from, and why you need it

Background & Context

If you have spent your first year of Python mostly with lists, dictionaries, and maybe a little NumPy, here is the mental gap that Pandas fills.

NumPy gives you the `ndarray`: a fast, homogeneous, N-dimensional grid of numbers. It is brilliant for math, but it has two blind spots that show up the moment you touch *real* data:

- **Positions, not labels.** A NumPy array is addressed by integer position (`a[0]`, `a[3]`). Real datasets are addressed by *meaning*: the row for "California," the column named "population."
- **One dtype, no gaps.** A NumPy array is a single data type with no natural notion of "missing." Real tables mix strings, floats, dates, and holes.

Pandas is a thin, powerful layer *on top of* NumPy that attaches *labels* to the axes of an array and gracefully handles missing values. Underneath, your numbers are still NumPy arrays (so you keep the speed); on top, you get a spreadsheet-like, database-like interface.

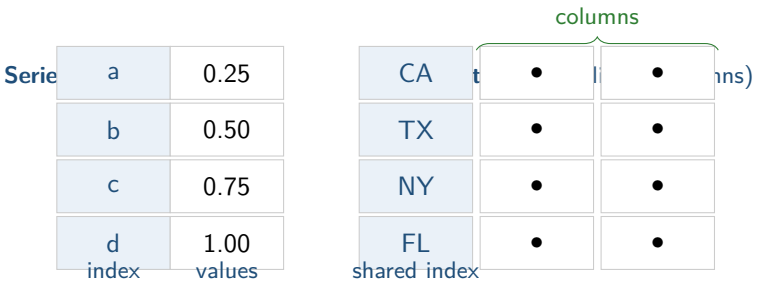
The convention everyone uses, and the one we use for the rest of these notes:

```
import numpy as np
import pandas as pd
```

Intuition

Think of the three core objects as a ladder of dimensionality, each one a labeled version of a NumPy array:

- **`Series`** = a 1-D NumPy array + an explicit index (labels for each element). Like a cross between a NumPy array and a Python dict.
- **`DataFrame`** = a 2-D table = an ordered collection of `Series` that all *share the same row index*. Like a dict of columns.
- **`Index`** = the immutable label array that both of the above use to align data.



2 The Series object

A `Series` is a one-dimensional array of *indexed* data. The simplest way to build one is from a list or NumPy array:

```
data = pd.Series([0.25, 0.5, 0.75, 1.0])
print(data)
```

```
0    0.25
1    0.50
2    0.75
3    1.00
dtype: float64
```

Notice that the printout has *two* columns: the index on the left (here the default integers `0..3`) and the values on the right. Those are exactly the two attributes you can pull out:

```
data.values   # → array([0.25, 0.5 , 0.75, 1.  ])  (a NumPy array)
data.index    # → RangeIndex(start=0, stop=4, step=1)
```

Key Idea

The `values` are a real NumPy array — that is where the speed lives. The `index` is a separate, *explicit* object. Standard NumPy arrays have only an *implicit* integer index; the whole point of a `Series` is that you can make the index whatever you want.

2.1 An index of your own choosing

Unlike a NumPy array, the index need not be integers, and need not start at zero:

```
data = pd.Series([0.25, 0.5, 0.75, 1.0],
                 index=['a', 'b', 'c', 'd'])
data['b']     # → 0.5
```

This is what people mean when they say a `Series` is "like a dictionary": you look values up by a label key. The difference from a plain Python `dict` is that the keys (the index) are a *typed, ordered* array, which makes slicing and vectorized math possible.

2.2 Series from a dictionary

Because a `Series` maps keys to values, building one directly from a `dict` is natural. Pandas sorts the keys into the index:

```
population_dict = {'California': 39_512_223,
                   'Texas':      28_995_881,
                   'New York':   19_453_561,
                   'Florida':    21_477_737}
population = pd.Series(population_dict)
```

```
California   39512223
Florida      21477737
New York     19453561
Texas        28995881
dtype: int64
```

Now it behaves like a dictionary — `population['California']` returns the value — *and* like an array, supporting slicing such as `population['California':'New York']`.

Common Pitfall

Label slicing includes the right endpoint. With the default integer positions, `data[0:2]` gives 2 elements (the usual Python "stop is exclusive" rule). But when you slice by *explicit labels*, e.g. `data['a':'c']`, Pandas includes `'c'`. Two different conventions live in the same object — this is exactly the ambiguity that `loc` and `iloc` (next section) exist to resolve.

3 The DataFrame object

If a `Series` is a labeled 1-D array, a `DataFrame` is a labeled 2-D array: a table whose rows share a common index and whose columns each have a name. Equivalently, and more usefully: *a* `DataFrame` *is a dictionary of* `Series` *that all share the same index.*

Suppose we have a second `Series` for area, indexed by the same states:

```
area = pd.Series({'California': 423_967, 'Texas': 695_662,
                  'New York': 141_297, 'Florida': 170_312})
states = pd.DataFrame({'population': population, 'area': area})
```

	population	area
California	39512223	423967
Florida	21477737	170312
New York	19453561	141297
Texas	28995881	695662

Like the `Series`, the `DataFrame` exposes its structure through attributes:

```
states.index     # the row labels: ['California','Florida','New York','
   Texas']
states.columns   # the column labels: ['population', 'area']
states.values    # the underlying 2-D NumPy array
```

Intuition

There are two ways to read a `DataFrame`, and you will switch between them constantly:

- **As a dict of columns:** `states['area']` returns the `area` `Series`. This is the *default* — a single key indexes a *column*, not a row. (This trips up people coming from NumPy, where `a[0]` is a row.)

- **As an enhanced 2-D array:** you can transpose it (`states.T`), do matrix-style math, and use the `loc`/`iloc` accessors for row/column selection.

3.1 Many ways to construct a DataFrame

You will meet all of these in the wild:

```
# 1. From a single Series (one column)
pd.DataFrame(population, columns=['population'])

# 2. From a list of dicts (each dict is a row)
pd.DataFrame([{'a': 1, 'b': 2}, {'a': 3, 'b': 4}])

# 3. From a dict of Series (each value is a column)  <- most common
pd.DataFrame({'population': population, 'area': area})

# 4. From a 2-D NumPy array, naming axes explicitly
pd.DataFrame(np.random.rand(3, 2),
             columns=['foo', 'bar'],
             index=['x', 'y', 'z'])
```

Common Pitfall

When you build from a *list of dicts* and the dicts have different keys, Pandas fills the missing entries with `NaN` rather than erroring. That is convenient but can silently hide a typo in a key name — always check `df.columns` after construction.

4 The Index object

Both `Series` and `DataFrame` contain an `Index`: a curious little object that is best thought of in two ways at once.

4.1 Index as an immutable array

You can slice and index it like a NumPy array, but you *cannot* mutate it:

```
ind = pd.Index([2, 3, 5, 7, 11])
ind[1]      # → 3
ind[::2]    # → Int64Index([2, 5, 11])
ind[1] = 0  # TypeError: Index does not support mutable operations
```

Background & Context

Why on earth make the index immutable? Because several `Series`/`DataFrame` objects can *share* the same index, and many operations *align* on it. If you could mutate an index in place, you could silently corrupt every object that shares it — like changing a primary key in a database while other tables still reference the old value. Immutability makes sharing safe and

makes alignment (the topic of section 3.3) predictable.

4.2 Index as an ordered set

Pandas indices support set operations, which is what makes joining and aligning datasets possible:

```
indA = pd.Index([1, 3, 5, 7, 9])
indB = pd.Index([2, 3, 5, 7, 11])
indA.intersection(indB)   # → [3, 5, 7]
indA.union(indB)          # → [1, 2, 3, 5, 7, 9, 11]
indA.symmetric_difference(indB)  # → [1, 2, 9, 11]
```

Key Idea

Keep this picture in your head for the rest of Chapter 3: **every Pandas operation that combines data — arithmetic, merging, grouping — works by lining up *labels* via these `Index` objects, not by lining up *positions*.** Once labels drive everything, "missing on one side" becomes a normal, expected case rather than an error, which is why missing-data handling (section 3.4) is woven so deeply into the library.

Section recap

- Pandas = NumPy speed + labeled axes + missing-data support.
- `Series`: 1-D values + an explicit `Index`; behaves like a dict *and* an array.
- `DataFrame`: aligned columns sharing one row index; behaves like a dict of `Series`. A bare key selects a *column*.
- `Index`: immutable, set-like; it is the machinery that aligns data across objects.
- Watch the slicing rule: label slices include the endpoint, positional slices do not.